

Redesigning a Cross-Platform App with Multi-Agent AI in One Day

How parallel AI agents in Cursor shipped a coordinated web + iOS home-page redesign with full test coverage

Sajad Paracha

June 6, 2026

Prepared for LinkedIn

How I used parallel AI agents in Cursor to ship a coordinated web and iOS home-page redesign - without sacrificing quality or test coverage.

The Goal

I run Outfit Suggestor, a production app that helps people build better outfits from photos of their clothes. The home experience needed a refresh: a dark navy aesthetic, a blue-to-purple brand gradient (#4facfe -> #c471ed), clearer navigation, and visual parity between React (web) and SwiftUI (iOS).

The catch: these are two different codebases, two design systems, and dozens of touchpoints. A single monolithic AI session would context-switch constantly and likely miss platform-specific details.

So I tried something different: multi-agent orchestration.

What Is Multi-Agent Development?

In Cursor, you can launch specialized agents that work in parallel on separate scopes. Instead of one assistant trying to do everything sequentially, you define focused missions:

Agent	Scope	Mission
Web agent	<code>`frontend/`</code>	Redesign home layout, components, Tailwind theme
iOS agent	<code>`ios-client/`</code>	Mirror the design in SwiftUI, update <code>`AppTheme`</code> ,

Each agent operates with a narrow context window tied to its platform. That reduces hallucinated cross-platform assumptions and speeds up delivery.

What We Shipped Today

1. Parallel Home-Page Redesign

Web (React 19 + TypeScript + Tailwind)

- New ``NavBar``, ``Sidebar``, ``OutfitPreview``, ``HowItWorksStepper``, ``RecentLooksSection``, and ``Footer``
- Brand tokens centralized in ``tailwind.config.js`` and ``index.css`` (``.btn-brand``, ``.bg-brand-gradient``)
- Filter controls rebuilt as visible grid ``<select>`` elements (replacing hidden pill UI that obscured values)

iOS (SwiftUI)

- ``MainFlowView``, ``MainTabView`` (4 tabs), ``HomeHeaderView``, ``RecentLooksSection``
- ``AppTheme`` aligned to the same navy + gradient palette
- Native carousel patterns and gold accent tags matching the mockup

Both agents worked from the same design reference - dark navy cards, gradient CTAs, instructional copy, and a polished mobile-first layout.

2. Design System Unification

After the parallel pass, a follow-up iteration hunted down legacy teal/green (#00B3A4, emerald, green-500) still leaking into Wardrobe, auth screens, and buttons. Everything was remapped to the brand gradient so the app feels like one product, not a patchwork.

3. Asset Pipeline from a Mockup

We cropped product photos directly from a reference screenshot using automated boundary detection (full-height gutter analysis + padding), bundled them for web (/public/examples/) and iOS (Assets.xcassets), then later removed the "Try these examples" section when the UX direction changed - a reminder that AI-assisted work is fast to build and fast to iterate.

4. Verification, Not Vibes

Every major change was validated:

- ****140/140**** frontend Jest tests passing
- ****16/16**** iOS tests passing
- Clean Xcode build on simulator

Multi-agent speed only matters if the result is merge-ready. Tests were the contract between agents and production.

Why Multi-Agent Beats Single-Agent Here

1. True parallelism

Web and iOS don't block each other. Two specialists finish in roughly the time one generalist would spend on half the work.

2. Platform-native output

The iOS agent thinks in SwiftUI modifiers and asset catalogs; the web agent thinks in Tailwind utilities and React composition. No awkward "translate this React component to Swift" mid-stream.

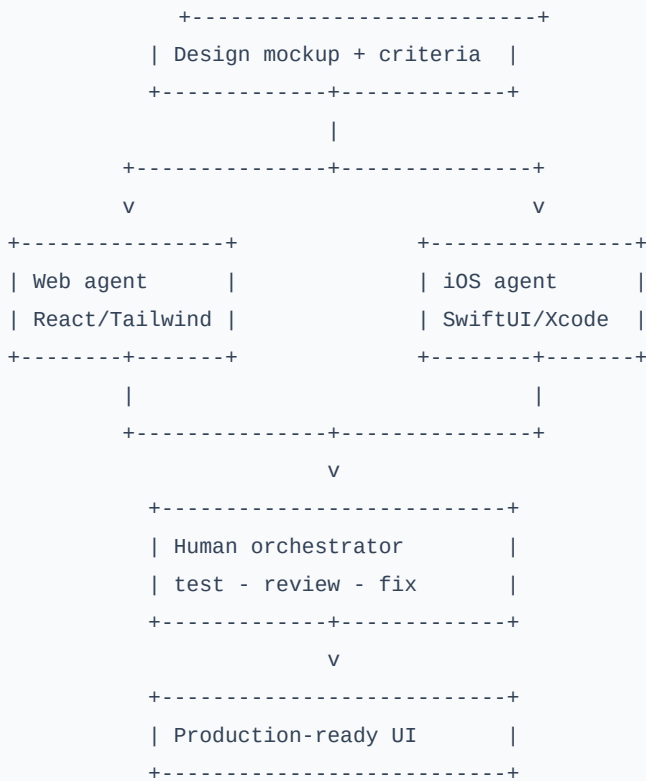
3. Smaller diffs, easier review

Each agent's changes cluster in one directory. Reviewers (human or AI) can approve platform slices independently.

4. Orchestrator role for the human

I stayed in the architect / QA seat: set the mockup, define acceptance criteria, run tests, and steer follow-ups (color cleanup, filter UX, section removal). The agents executed; I integrated.

Workflow Diagram



Lessons for Teams Adopting Multi-Agent AI

1. Scope agents narrowly. "Redesign the iOS home tab" beats "fix the app."
2. Share one source of truth. A mockup image + written tokens (#4facfe, #c471ed, navy #0f172a) kept both agents aligned.
3. Automate the gate. Tests are how you trust parallel workstreams.
4. Plan for iteration. We added, refined, and removed features in the same session - velocity includes undoing the wrong thing quickly.
5. You are still the tech lead. Agents propose; you prioritize, merge, and decide when "done" means done.

Stack

- **Backend:** FastAPI (port 8001)
- **Web:** React 19, TypeScript, Tailwind CSS, Jest
- **iOS:** SwiftUI, MVVM, XCTest / XCUITest
- **AI tooling:** Cursor IDE with parallel Task agents

Live app: <https://sajadparacha.github.io/outfit-suggestor-app>

Closing Thought

Multi-agent AI doesn't replace engineering judgment - it compresses the execution layer. The interesting work shifts to defining problems, splitting them cleanly, and verifying outcomes. For cross-platform products, that split is natural: one agent per platform, one human orchestrating the symphony.

If you're experimenting with agentic development, start with a bounded redesign where platforms are independent but design is shared. Measure with tests. Ship the same day.

****#AI #Cursor #MultiAgent #SoftwareEngineering #SwiftUI #React #CrossPlatform #FashionTech #DeveloperProductivity #AgenticAI****

Sajad Paracha - Outfit Suggestor